

Project Executable Files

Date	01-07-2026
Team ID	SWTID-2026-3845
Project Name	SMART LENDER
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

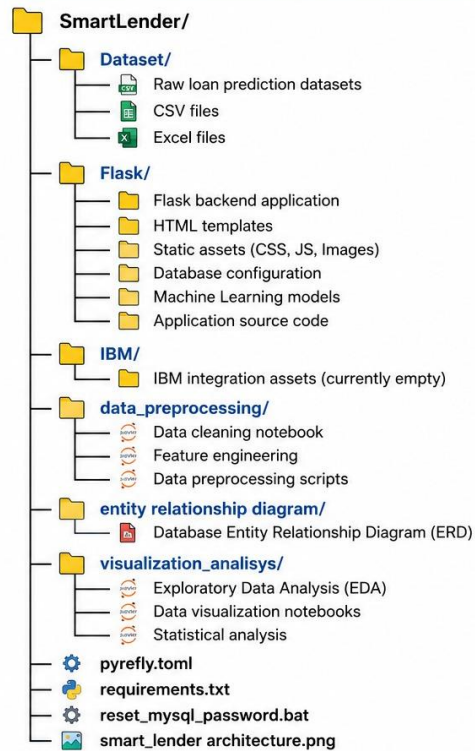
Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes/ No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	No
8	Dockerfile / Containerization config (if applicable)	No
9	Test files and test results	Yes

10	Demo video or walkthrough (if required)	Yes
----	---	-----

Step 2: File / Folder Structure

SmartLender – Project Folder Structure



Folder / File	Description
Dataset/	Raw loan prediction datasets (CSV & Excel)
Flask/	Flask web application, ML models, templates, static files, and database configuration
IBM/	IBM integration assets (currently empty)
data_preprocessing/	Data cleaning and preprocessing notebooks
entity relationship diagram/	Database Entity Relationship Diagram (ERD)
visualization_analysys/	Exploratory Data Analysis (EDA) and visualization notebooks
pyrefly.toml	Python project configuration file
requirements.txt	Python package dependencies
reset_mysql_password.bat	Utility script to reset the local MySQL password
smart_lender architecture.png	High-level system architecture diagram

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://smart-lender-dple.onrender.com
Login Credentials (Demo)	None
Platform / Hosting Provider	Render
Repository Link	https://github.com/kaladharroyal/smart-lender
Demo Video Link	

Pre-requisites:

- Python 3.8 or higher
- pip (Python package manager)
- Git
- Web browser (Chrome/Firefox)
- Required libraries: Flask, Pandas, NumPy, Scikit-learn, XGBoost, Pickle/Joblib

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:

Python 3.8 - 3.12, MySQL Server (v8.0 or newer), pip (Python package manager).

Step 1: Create a virtual environment: `python -m venv .venv`

Step 2: Activate the virtual environment (Windows PowerShell): `.venv\Scripts\Activate.ps1`

Step 3: Install dependencies: `pip install -r requirements.txt`

Step 4: Log in to MySQL and create the database: `CREATE DATABASE IF NOT EXISTS smart_lender;`

Step 5: Import the database schema: `mysql -u root -p smart_lender < Flask/schema.sql`

Step 6: Create a `.env` file in the root directory (use `.env.example` as reference):

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_mysql_password
DB_NAME=smart_lender
```

Step 7: Run the Flask application: `python Flask/app1.py`

Step 8: Open browser at `http://127.0.0.1:5000/`

Step 5: Known Issues / Limitations

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Model test accuracy is capped at ~84.9% (XGBoost); some misclassifications occur for borderline applicants.	Acceptable for demo/prototype use; improvement would need more data and feature engineering.
2	Dataset is small (614 records) and balanced using SMOTE, which generates synthetic samples rather than real data.	May not generalize to larger real-world populations; retraining on live data recommended before production.
3	App is hosted on Render's free tier, which spins down when idle, causing a cold-start delay on first request.	Acceptable for demo; a paid/always-on tier would resolve this in production.
4	No user authentication/login system; anyone with the URL can submit applications or view the dashboard.	Suitable for demo/evaluation only; role-based access control needed before real deployment.
5	MySQL database is a single instance with no replication or backup strategy.	Fine for prototype; production use needs managed DB hosting with backups.
6	Outlier handling (IQR) and missing-value imputation (mode) are simple statistical methods.	Works for current dataset; more advanced techniques could be explored in future iterations.